

Homework Assignment 7

This document contains the write-ups for the three challenges of **Assignment 7**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	spec_ops	<code>fsuCTF{b14ck_bel7_in_blackb0x}</code>	- Create a dummy <code>input.txt</code> file with 32 characters to satisfy the initial file check. - Run the binary in GDB and set a breakpoint after the <code>loadData</code> function. - Manually overwrite the <code>RAX</code> register to <code>0</code> to bypass error handling and force program execution. - Advance to the <code>checkState</code> function and extract the encrypted flag and the generated key from memory buffers. - XOR the extracted key with the target buffer in reverse order to recover the plaintext flag.
Problem 2	CoffeeGrinder	<code>fsuCTF{1m_g0nn4_n33d_n3w_k1dn3y5}</code>	- Unzip the <code>CoffeeGrinder.jar</code> file to extract the compiled Java classes and the encrypted binary. - Decompile the <code>.class</code> files using <code>jadx</code> to analyze the source code and encryption logic. - Identify the encryption parameters: <code>AES</code> in <code>ECB</code> mode with the password "<code>adenosine</code>". - Write a Python script to replicate the key generation using a <code>SHA-256</code> hash of the password. - Decrypt the <code>encrypted_secret.bin</code> file and remove the padding to reveal the flag.
Problem 3	Chicken Scratch	<code>fsuCTF{7hi5_Is_Ju5t_Chick3n_Scr47ch}</code>	- Identify the ELF file as a bundled Python program created with <code>PyInstaller</code>. - Extract the contents of the executable using <code>PyInstxtractor</code> to obtain the compiled <code>.pyc</code> file. - Generate a disassembly of the Python bytecode using <code>pycdas</code> since traditional decompilers fail for version 3.10. - Analyze the bytecode to map the symbol strings to their corresponding alphanumeric characters. - Automate the translation of the challenge list (variable <code>BBBB</code>) using the identified substitution dictionary to reconstruct the flag.

PROBLEM 1 - spec_ops

1. Analysis

To begin with I started by checking the file type and running it to see the behavior described in the prompt.

```
L$ file spec_ops
spec_ops: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=d35f602121e313a75ce20e3806806fb2705bdd5d, for GNU/Linux 4.4.0, not stripped
```

The file is **not stripped**, which means the function names and symbols are preserved, making our analysis easier.

When I executed it, the program indeed terminated immediately with no output. To understand why it was quitting, I used `ltrace` to see library calls:

```
ltrace ./spec_ops
malloc(24) = 0x558ad2c3d310
malloc(56) = 0x558ad2c3d330
fopen("./input.txt", "r") = nil
free(nil) = <void>
free(nil) = <void>
free(0x558ad2c3d310) = <void>
free(0x558ad2c3d330) = <void>
+++ exited (status 1) +++
```

The program tries to open a file named `input.txt`. Since it didn't exist, `fopen` returned `nil`, and the program safely exited.

I then loaded the binary into GDB to examine the assembly. I disassembled the `main` function using the command:

```
gef> disas main
```

Here are the critical sections I identified:

1. Between `<+139>` and `<+191>`, the program moves four 64-bit constants into the stack:

```
0x00000000000014eb <+139>: movabs rax, 0x70282ce093c8b54d
0x00000000000014f5 <+149>: movabs rdx, 0x45cef3f733855442
0x00000000000014ff <+159>: mov     QWORD PTR [rbp-0x30], rax
0x0000000000001503 <+163>: mov     QWORD PTR [rbp-0x28], rdx
0x0000000000001507 <+167>: movabs rax, 0xe62ba16c7a2a45ce
0x0000000000001511 <+177>: movabs rdx, 0x2017446a30c096fe
0x000000000000151b <+187>: mov     QWORD PTR [rbp-0x22], rax
0x000000000000151f <+191>: mov     QWORD PTR [rbp-0x1a], rdx
```

This looks like the encrypted flag, totaling 32 bytes of data.

2. At `<+371>`, it calls `makeKey` after setting `esi` to `0x2a` (42 in decimal).

```
0x00000000000015c1 <+367>: mov     rax, QWORD PTR [rbp-0x38]
0x00000000000015d3 <+371>: mov     esi, 0x2a
0x00000000000015d8 <+376>: mov     rdi, rax
0x00000000000015db <+379>: call    0x1402 <makeKey>
0x00000000000015e0 <+384>: mov     rax, QWORD PTR [rbp-0x38]
```

`0x2a` might be the seed used to generate a "random" key. Since it is constant, the generated will be identical every time we run the program.

I disassembled the `makeKey` function to verify this, and confirmed that the C function `srand` is called with the value `0x2a`, which is passed as an argument from `main`.

```
0x000000000000140e <+12>: mov     DWORD PTR [rbp-0x1c], esi
0x0000000000001411 <+15>: mov     eax, DWORD PTR [rbp-0x1c]
0x0000000000001414 <+18>: mov     edi, eax
0x0000000000001416 <+20>: call    0x1090 <srand@plt>
```

3. The `main` function executes a sequential chain of critical operations:

`loadData` → `makeKey` → `performEncryption` → `checkState`

Each function must return a success signal for the next one to execute. If any function encounter an error, the program breaks the chain and terminates immediately. It does this by inspecting the `EAX` register immediately after every function call using always the following set of instructions, to skip the error-handling block when a function successfully returns 0.

```
call    <function_name>
test    eax, eax                ; Sets the Zero Flag if EAX is 0
je      <success_label>        ; Jumps to the next task if EAX was 0
...     ; Logic to set error code and exit
```

2. Solution Strategy

Based on the disassembly, my strategy was:

1. Bypass the File Check:

Create a dummy `input.txt` with 32 characters to satisfy the `loadData` function.

2. Dynamic Flow Manipulation:

Use GDB to reach `checkState`. If the program tries to exit early due to my dummy input, I will manually overwrite the `RAX` register to "force" a success state.

3. Memory Extraction:

Once inside `checkState`, I will inspect the registers.

- `$rdi` should hold my processed input
- `$rsi` should hold the actual encrypted flag

4. XOR Recovery:

Since the encryption key is likely deterministic (due to the `0x2a` seed), I will find where the key is stored in memory and XOR it with the encrypted flag to get the plaintext.

3. Execution and Flag

1. Step 1: Satisfying `loadData`

I created an input file (`input.txt`) with 32 "A"s.

2. Step 2: Forcing the Execution Path

I ran the program in GDB and set a breakpoint right after `loadData` at `*main+323`.

```
gef> break *main+323
Breakpoint 1 at 0x15a3
gef> run
```

The program succesfully loaded our input in `$rsi`:

```
gef> x/10i $rsi
0x0000000000000000: 0x0000000000000000
0x0000000000000001: 0x0000000000000000
0x0000000000000002: 0x0000000000000000
0x0000000000000003: 0x0000000000000000
0x0000000000000004: 0x0000000000000000
0x0000000000000005: 0x0000000000000000
0x0000000000000006: 0x0000000000000000
0x0000000000000007: 0x0000000000000000
0x0000000000000008: 0x0000000000000000
0x0000000000000009: 0x0000000000000000
```

The program stopped. I checked `RAX` and saw it was non-zero (indicating an error).

```
$rax: 0x1
```

I forced it to zero so the program would continue:

```
gef> set $rax = 0
gef> break checkState
Breakpoint 2 at 0x5555555538e
gef> continue
```

3. Step 3: Finding the Key and the Target

The program stopped at `checkState`. I checked the arguments:

- `$rsi` points to the **Encrypted Flag**
- `$rdi` points to a structure. By inspecting `$rdi`, I found it contained a pointer to a second buffer at `0x55555555a5b0`.

I extracted both buffers:

```
gef> x/32xb $rsi
0x7fffffffda00: 0x4d  0xb5  0xc8  0x93  0xe0  0x2c  0x28  0x70
0x7fffffffda08: 0x42  0x54  0x85  0x33  0xf7  0xf3  0xce  0x45
0x7fffffffda10: 0x2a  0x7a  0x6c  0xa1  0x2b  0xe6  0xfe  0x96
0x7fffffffda18: 0xc0  0x30  0x6a  0x44  0x17  0x20  0x00  0x00
gef> x/32xb 0x55555555a5b0
0x55555555a5b0: 0x13  0xf1  0x30  0xcd  0xf8  0xf1  0x8b  0x4f
0x55555555a5b8: 0x49  0x41  0x20  0x0b  0xeb  0x5a  0xa8  0xc4
0x55555555a5c0: 0xa2  0x20  0x48  0x25  0x07  0xc2  0x1f  0xd7
0x55555555a5c8: 0x9c  0xed  0x86  0x64  0x29  0x31  0x64  0x46
```

4. Step 4: Solving the XOR

At first, XORing the contents of both buffers did not seem to produce any meaningful result. However, I noticed that when XORing the last bytes of each buffer, the flag comparison was being performed in reverse order.

- `0x20` (Target) $\oplus$ `0x46` (Key) = `0x66` ('f')
- `0x17` (Target) $\oplus$ `0x64` (Key) = `0x73` ('s')
- `0x44` (Target) $\oplus$ `0x31` (Key) = `0x75` ('u')

I wrote a Python script to automate the 30-byte XOR (*not 32 because last two positions are 0*):

```
target = [
    0x4d, 0xb5, 0xc8, 0x93, 0xe0, 0x2c, 0x28, 0x70,
    0x42, 0x54, 0x85, 0x33, 0xf7, 0xf3, 0xce, 0x45,
    0x2a, 0x7a, 0x6c, 0xa1, 0x2b, 0xe6, 0xfe, 0x96,
    0xc0, 0x30, 0x6a, 0x44, 0x17, 0x20
]

key = [
    0x13, 0xf1, 0x30, 0xcd, 0xf8, 0xf1, 0x8b, 0x4f,
    0x49, 0x41, 0x20, 0x0b, 0xeb, 0x5a, 0xa8, 0xc4,
    0xa2, 0x20, 0x48, 0x25, 0x07, 0xc2, 0x1f, 0xd7,
    0x9c, 0xed, 0x86, 0x64, 0x29, 0x31, 0x64, 0x46
]

flag = ""

for i in range(1, 31):
    flag += chr(target[-i] ^ key[-i])

print(flag)
```

Flag: `fsuCTF{b14ck_bel7_in_blackb0x}`

PROBLEM 2 - CoffeeGrinder

This challenge just provides us with a file named `CoffeeGrinder.jar`.

1. Analysis

Since a JAR is essentially a ZIP file containing Java bytecode, I decided to unzip it to see the "parts" of this coffee grinder.

```
└─$ unzip CoffeeGrinder.jar -d ./extracted
Archive:  CoffeeGrinder.jar
  creating: ./extracted/META-INF/
  inflating: ./extracted/META-INF/MANIFEST.MF
  inflating: ./extracted/CryptUtils.class
  inflating: ./extracted/Main.class
  inflating: ./extracted/encrypted_secret.bin
```

- `Main.class` & `CryptUtils.class`: Are the compiled java classes
- `encrypted_secret.bin`: Looks like the encrypted flag

I used `jadx` to decompile the `.class` files to read the source code, using the command:

```
$ sudo jadx ../../CryptUtils.class -d ../../source_code
```

After decompiling both classes, we obtain a directory structure for each class containing their Java source code.

```
└─
└─ resources
└─ sources
└─ depackage
└─ CryptUtils.java
```

Once decompiled, I looked at the source code.

Main.java

```
public class Main {
    public static void main(String[] strArr) {
        try {
            System.out.println("Encrypting file...");
            CryptUtils.encryptFile("adenosine", "flag.txt", "encrypted_secret.bin");
            System.out.println("Success! File encrypted to: " + "encrypted_secret.bin");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

This `Main` class is actually a tool to *create* the challenge. It takes a file called `flag.txt`, uses a secret string `"adenosine"`, and produces `encrypted_secret.bin`. To get the flag, I need to do the exact opposite.

CryptUtils.java

```
public class CryptUtils {
    private static final String ALGORITHM = "AES";
```

```

private static final String TRANSFORMATION = "AES/ECB/PKCS5Padding";

public static void encryptFile(String str, String str2, String str3) throws Exception {
    SecretKeySpec secretKeySpec = new SecretKeySpec(
        MessageDigest.getInstance("SHA-256").digest(str.getBytes("UTF-8")),
        TRANSFORMATION
    );
    Cipher cipher = Cipher.getInstance(TRANSFORMATION);
    cipher.init(1, secretKeySpec); // 1 means ENCRYPT_MODE
    // ... reads flag.txt and writes to encrypted_secret.bin ...
}
}

```

This code **encrypts a file using AES encryption**.

1. It first derives an AES key by hashing a password with SHA-256
2. Then initializes an AES cipher in ECB mode with PKCS5 padding
3. It finally encrypts the contents of a file (most likely containing the flag) and writes the encrypted output to the file `encrypted_secret.bin`

2. Solution Strategy

What we need to retrieve the flag encrypted in `encrypted_secret.bin` is a decryption script. To build it, I had to understand the cryptographic choices made by the author:

The Key:

The code first uses **SHA-256**.

This hashing algorithm turns any text into a 32-byte sequence. Here it is used to generate an AES key from the password "adenosine".

AES:

A **symmetric** block cipher.

Mode: ECB:

This mode doesn't use an Initialization Vector (IV), which means every time you encrypt the same block of data with the same key, you get the same output.

Padding: PKCS5:

Since AES works on 16-byte blocks, if the flag isn't a perfect multiple of 16, the program adds "padding" to fill the gap.

To solve this challenge we will design a script that:

1. **Replicates the key generation** by hashing the password "adenosine" with **SHA-256**.
2. **Reads the encrypted data** from the file `encrypted_secret.bin`
3. **Initializes an AES cipher in ECB mode** using the derived key
4. **Decrypts the ciphertext and removes the padding** to recover the original plaintext

3. Execution and Flag

I wrote a Python script using the `pycryptodome` library to perform the decryption.

```

import hashlib
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

# 1: Replicate the key generation
password = "adenosine"

```

```
key = hashlib.sha256(password.encode('utf-8')).digest()

# 2: Load the encrypted "part"
with open("encrypted_secret.bin", "rb") as f:
    ciphertext = f.read()

# 3: Setup AES in ECB mode
cipher = AES.new(key, AES.MODE_ECB)

# 4: Decrypt and unpad
decrypted = unpad(cipher.decrypt(ciphertext), AES.block_size)
print(f"Flag: {decrypted.decode('utf-8')}")
```

After running the script it prints the flag in the terminal.

Flag: `fsuCTF{1m_g0nn4_n33d_n3w_k1dn3y5}`

PROBLEM 3 - Chicken Scratch

In this challenge we are provided with a single binary file: `chicken_scratch`.

1. Analysis

I started by checking what kind of file I was dealing with using the `file` command:

```
❯$ file chicken_scratch
chicken_scratch: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=3d4132e6aade8fe199dfd898058aa768ebd7f197, stripped
```

The output indicates that the file is a stripped ELF binary, meaning the author removed the symbol information, so function names are not visible

I tried to execute the file to see its behavior, but I ran into a library error:

```
❯$ ./chicken_scratch
[PYI-3787:ERROR] Failed to load Python shared library '/tmp/_MEIW7uVbM/libpython3.10.so.1.0': libcrypt.so.2: cannot open shared object file: No such file or directory
```

The prefix `[PYI-...]` and the mention of `libpython3.10` reveal that this isn't a native C/C++ program. It is a **Python program** packaged as a standalone executable using PyInstaller.

After some research, I found that PyInstaller bundles the Python interpreter, the script, and all its dependencies into a single binary. When executed, it extracts them to a temporary directory and runs the script.

Since it's a PyInstaller bundle, my goal shifted from reversing a binary to decompiling a python program.

I used **PyInstxtractor** to pull the original files out of the ELF wrapper:

```
$ python3 pyinstxtractor.py chicken_scratch
```

Looking into the extracted folder, I found several files, but one stood out: `chicken_scratch.pyc`.

A `.pyc` file is **compiled Python bytecode**. It's not human-readable source code. I tried to decompile it back to `.py` using the `pycdc` decompiler (*I had to clone it and compile it from [github](#)*) but I got an error:

⚡ Error when trying to decompile

```
❯$ ./pycdc ../chicken_scratch_extracted/chicken_scratch.pyc
# Source Generated with Decompyle++
# File: chicken_scratch.pyc (Python 3.10)

Unsupported opcode: MAP_ADD (188)
# WARNING: Decompyle incomplete
```

I also tried other decompilation tools such as `uncompyle6` and `decompyle3`, but they failed for the same reason: the `.pyc` file was compiled with **Python 3.10**, and the tools I tested do not fully support that bytecode version, so the script cannot be decompiled.

If I couldn't get the source code, I had to read the "assembly" of Python. I used `pycdas` to generate a disassembly of the bytecode.

```
$ pycdas chicken_scratch.pyc > disassembly.txt
```

The output file is a txt with the bytecode disassembly of the script, showing the low-level instructions executed.

Bytecode Dissassembly breakdown

The file is divided into several sections:

- **[Names]**
 - A list of variable names (`AAAA`, `BBBB`, `CCCC`, `DDDD`) and built-in functions (`input`, `exit`, `print`) used as identifiers.
- **[Constants]**
 - A large repository of static data, formed by
 - Long strange sequences of symbols
 - The alphanumeric characters (`0-9`, `a-z`, `A-Z` and `symbols`) they map to .
- **[Dissassembly]**
 - Operations executed. It divides into four distinct phases of execution
 1. **Dictionary Initialization (AAAA)**

The program builds a map used as a translation table.

 1. First it initializes the dictionary with `0 BUILD_MAP 0`
 2. For each entry it does the mapping like shown above:

```
2      LOAD_CONST      0: ' /+--_-\|/|<|\\+<<|\\>|\\-/
4      LOAD_CONST      1: '0'
6      MAP_ADD          1
```

3. Saves the dictionary as AAAA with `104 STORE_NAME 0 (AAAA)`
2. **Challenge List Creation (BBBB)**

The program prepares the sequence of "symbol strings" the user must decode .

 - `568 BUILD_LIST 0`: Creates an empty list.
 - `570 LOAD_CONST 176`: Loads a large tuple containing the specific scratch sequences for the challenge.
 - `572 LIST_EXTEND 1`: Appends all elements from the tuple into the list.
 - `574 STORE_NAME 2 (BBBB)`: Saves the list as `BBBB` .

3. The Validation Loop

The program iterates through the `BBBB` list and checks user input against the translation map .

- `580 FOR_ITER` : Iterates over each scratch sequence (`CCCC`) in the list.
- `584 LOAD_NAME 4 (input)` : Prompts the user to enter a value for the current scratch string .
- `600 LOAD_NAME 0 (AAAA)` : Accesses the translation dictionary.
- `604 BINARY_SUBSCR` : Retrieves the **correct** character for the current scratch from `AAAA` .
- `606 COMPARE_OP 3 (!=)` : Compares the user's input (`DDDD`) with the correct character.
- `608 POP_JUMP_IF_FALSE` : If the input is correct (it is **not** different), it continues to the next item.
- `612 LOAD_NAME 6 (exit)` : If the input is incorrect, the program terminates immediately.

4. Success Message

Final instructions after all challenges are passed.

- `622 LOAD_NAME 7 (print)` : Calls the print function.
- `624 LOAD_CONST 178` : Prints the message: "You decoded all the chicken scratch but where is the flag?".
- `632 RETURN_VALUE` : Exits the program.

2. Solution Strategy

After analyzing the bytecode, we can infer that the original Python script would look something like this:

```
# AAAA -> dictionary mapping long "scratch" strings to simple characters
AAAA = {
    '/+--_-\\|/<\\|+<>\\|>-\\|-/<_<_<\\|_\\|/<\\|\\|_+>>/_<_\\|_+/_+>-_/+\\|\\|/<+>-/-
    |_//>/<>\\|\\|\\|_-->>|/_\\|/_><-__': '0',

    '><-_|++/-\\|_>_<_+|>+<_/_-<>\\|<->+_-/+--</-\\|_|+_+</---<<+\\|+\\|>+\\|+--</<<-
    |_//|>+//<|_+\\|>|/_<\\|_->|': '1',

    # ... More mappings for a-z, A-Z, and symbols
}

# BBBB -> a list containing the sequence of scratches the user must 'translate'
BBBB = [
    '|-\\|_><_>+</|<+\\|/_>-+\\|>\\|\\|+||_-->>/_+\\|>|+_<+|\\|/_>/<-_>///+_\\|<<\\|\\|>+>/\\|_-+
    +-/_>\\|/<->///<\\|\\|+>_>|\\|_-<|>\\|\\|',

    '_/_>_>->|_//>_>>\\|>+\\|<+|>|\\|/_+>+/-<<+>|_<<+/_+/-<>_<_</|\\|_|<-/_\\|><\\|\\|+<\\
    \\|_-\\|+<>_+>+//|_+>>/-|/_<',

    # ... (The full list of sequences to be guessed)
]

# The program iterates through the list of challenges
for CCCC in BBBB:
    # Displays the scratch and asks for the translation
    DDDD = input(f"{CCCC}: ")

    # Compares the input with the correct value stored in dictionary AAAA
    if DDDD != AAAA[CCCC]:
        exit() # If you get one wrong, the program exits immediately

# If you complete them all correctly
print("You decoded all the chicken scratch but where is the flag?")
```

The program just holds a list of 104 scratches in the variable `BBBB` and asks the user to provide the correct "translation" for each one by looking it up in the dictionary `AAAA` . If the user provides the correct character for every single scratch in the sequence, the program succeeds

To confirm this theory, I performed a manual lookup. By searching for the symbol strings from the `BBBB` list within the dictionary `AAAA`, the translation began to take shape:

- First "scratch" in list 176 → maps to `f`.
- Second "scratch" in list 176 → maps to `s`.
- Third "scratch" in list 176 → maps to `u`.

I could see the string `fsu` forming, which matches the expected flag format.

3. Execution and Flag

To solve it, I wrote a small Python script to automate the translation:

```
import re

def solve():
    data = open("disassembly.txt").read()

    # Extract constants
    consts = {m.group(1): m.group(2) for m in re.finditer(r"(\d+):\s+'(.*?)'", data)}

    # Build mapping from LOAD_CONST -> LOAD_CONST -> MAP_ADD
    mapping = {}
    last = penultimate = ""

    for line in data.splitlines():
        if "LOAD_CONST" in line:
            m = re.search(r"LOAD_CONST\s+(\d+)", line)
            if m:
                penultimate, last = last, m.group(1)

        elif "MAP_ADD" in line and penultimate in consts and last in consts:
            mapping[consts[penultimate]] = consts[last]

    # Decode constant 176
    block = data.split("176:")[1].split("STORE_NAME")[0]
    scratches = re.findall(r"'(.*?)'", block)

    decoded = "".join(mapping.get(s, "?") for s in scratches)
    print("Decoded:", decoded)

solve()
```

After translating all 104 characters in the sequence, we get the output:

```
Good job reversing this! I hope everyone is enjoying the lessons.
---
fsuCTF{7hi5_Is_Ju5t_Chick3n_Scr47ch}
---
When we get to cryptography we will talk about this kind of program again because this is a
substitution cipher!
```

Flag: `fsuCTF{7hi5_Is_Ju5t_Chick3n_Scr47ch}`